Does Incremental Polynomial Feature Expansion Help Class-Incremental Learning?

A Controlled Comparison with Experience Replay on Split MNIST

Mohamed Alkobrosli

Draft manuscript

Abstract

Class-incremental learning (CIL) asks a single model to absorb new classes over time without erasing old ones. A recent line of work proposes to grow a model's capacity *explicitly*: start from a low-degree polynomial feature map, append zero-initialised higher-degree monomials as classes arrive, and protect old knowledge with replay and per-coefficient update rules. We ask whether this works, why, and at what cost. Under one protocol (Split MNIST, five two-class experiences, a fixed 16-dimensional random projection, 200 replay exemplars per class, three epochs, identical experiences and replay exemplars per seed) we compare the polynomial method, with settings chosen on a validation split from 216 candidate configurations, against an experience-replay MLP (ER-MLP) tuned on 35 configurations. The validation-selected polynomial setting reaches 77.8% final accuracy (forgetting 10.5%), against 75.7% (17.0%) for the validation-selected ER-MLP, and 76.0% for the ER-MLP picked in hindsight on the test set. The paired accuracy difference is +1.8 points (95% CI +1.0 to +2.6) against that strongest ER-MLP, positive in all 7 seeds. Ablations, however, do not support the mechanism the method is built around. Most of the gain over a linear model comes from polynomial *capacity* (66.8% for degree 1 versus 77.5% for degree 3 from the start), not from expanding it gradually; class-balanced sampling matters far more than any update rule (median accuracy 77.1% versus 54.0% under shuffled sampling); and per-degree learning rates change class-balanced accuracy by at most +0.5 points. We also show that "lowest forgetting" and "smallest time" are degenerate objectives on their own (the winners learn almost nothing), and that the best *trade-off* configuration depends on an arbitrary weighting. All claims are limited to one dataset, one projection, one competitor family and seeds that vary only shuffling and replay selection.

1 Introduction

Neural networks trained on a stream of tasks tend to overwrite what they previously learned, a failure known as catastrophic forgetting [1]. In class-incremental learning (CIL) the difficulty is sharpest: the label set grows, no task identity is given at test time, and the model must separate classes that were never seen together [2, 3]. Replay of a small stored subset of earlier data is a strong and simple remedy [4, 5], and strong simple baselines are repeatedly found to be hard to beat [3, 6].

This paper evaluates a different route to the same goal. The *polynomial CIL* method studied here represents an input by its monomials up to a degree d and learns a linear softmax head on top. Because monomials of degree d are a prefix of those of degree $d+1$, the model can be enlarged by appending coefficients initialised to zero, which leaves every existing output *exactly* unchanged. The hypothesis behind the method is that such incremental expansion, combined with replay and update rules that protect old coefficients, lets a model learn new classes while retaining old ones.

The hypothesis is plausible but has several separable parts, and a favourable headline number does not tell us which part matters. We therefore organise the study around five questions.

RQ1 Under identical data, experiences, replay memory and training budget, does polynomial CIL outperform a standard experience-replay MLP in accuracy, forgetting and cost?

RQ2 Where does any advantage come from: polynomial capacity, gradual expansion, replay, the update rule, the sampling policy, or per-degree learning rates?

RQ3 What does "best" mean? How do configurations chosen for accuracy, for forgetting, for time, and for a trade-off differ, and are the extreme choices meaningful?

RQ4 Is the comparison stable to the training budget (3 versus 10 epochs)?

RQ5 Does selecting configurations on validation data transfer to the test set, and how large is the optimism of selecting on the test set?

Contributions. (i) A controlled, reproducible comparison in which both algorithms receive byte-identical experiences and replay exemplars (checked by plan fingerprints), with all hyper-parameters chosen on a validation split. (ii) An analysis tool that, from the study's raw results, selects the highest-accuracy, lowest-forgetting, smallest-time and best-trade-off configurations together with the parameters that define them. (iii) Evidence that the polynomial method is competitive with and modestly better than an ER-MLP on this protocol, together with ablations that attribute most of the effect to capacity and balanced replay rather than to expansion or update rules. (iv) An explicit account of limitations and of what would be needed to turn these findings into general claims.

2 Background and related work

CIL protocols and metrics. Scenario taxonomies distinguish task-, domain- and class-incremental learning [2, 3]. Evaluation commonly uses an accuracy matrix $R_{i,j}$ and summaries of final accuracy and forgetting [7, 8]; it has been argued that forgetting alone is misleading because a model that never learns anything new cannot forget [7, 9], and that evaluations must be robust to protocol choices [10]. We adopt both points: we report forgetting only next to final and newest-experience accuracy, and we state the protocol precisely.

Remedies for forgetting. Regularisation (EWC, synaptic intelligence, LwF) [11, 12, 13], replay and exemplar methods [5, 14, 4], and gradient-projection methods [15, 16] are the main families; surveys include [8, 17]. Two of the polynomial method's update rules ("Strategy B") constrain gradients to the null space of the design matrix at a few stored reference inputs, which is in the spirit of the projection methods.

Polynomial features. Explicit low-degree polynomial maps feeding a linear model are an old idea [18, 19], and polynomial networks have been revisited recently [20]. To our knowledge their use as an *incrementally growing* head for CIL is the specific proposal examined here; we do not survey whether others have studied it.

3 The method under study

Inputs and features. An MNIST image is normalised and mapped to $z \in \mathbb{R}^{16}$ by a fixed random orthonormal projection that is never fitted to data. The degree-d feature map is

$$\varphi_d(z) = \Big[\, 1;\;\; s_1\, m_1(z);\;\; s_2\, m_2(z);\; \ldots;\; s_d\, m_d(z) \Big], \qquad s_k = \binom{16+k-1}{k}^{-1/2}, \tag{1}$$

where $m_k(z)$ stacks all monomials of total degree k. The scaling s_k keeps high-degree groups from dominating merely by being numerous. The feature counts are 17, 153 and 969 for $d = 1, 2, 3$.

Data	MNIST (60 000 train, 10 000 test), archives verified by SHA-256
Input	$(x - 0.1307)/0.3081$, fixed orthonormal random projection to 16-D (seed 0)
Experiences	$\{0,1\},\{2,3\},\{4,5\},\{6,7\},\{8,9\}$; one 10-way head; future classes never compete
Budget	3 epochs per experience, batch 128 (10 epochs as a secondary study)
Memory	fixed random subset of 200 exemplars per class (1 600 stored at the end)
Validation	10% hold-out of the training set (seed 123), used only for selection
Seeds	selection seeds 0–2 (validation); evaluation seeds 3–9 (test); disjoint
Metrics	$R_{i,j}$ accuracy on experience j after learning i; final accuracy, forgetting

Table 1: Split MNIST CIL protocol used for every algorithm.

Model, prediction and expansion. The head is $W \in \mathbb{R}^{p\times 10}$ with logits $\varphi_d(z)^\top W$ and no separate bias (the constant monomial plays that role). Only columns of classes introduced so far take part in the softmax, in training and in prediction, so a future class can never be predicted. Starting at degree 1 the degree schedule is $(1,2,3,3,3)$ over the five experiences. Expansion appends zero rows to W; the core implementation asserts that the logit change is below 10^{-5}.

Training. Each experience is trained with mini-batch SGD on cross-entropy over the current experience plus a fixed random replay memory of 200 examples per earlier class. *Sampling* is either `shuffled` (current and replay data shuffled together) or `class_balanced` (every batch contains almost equal numbers of every class seen so far; a class with fewer examples than its share is repeated, never padded from another class). *Update rules* decide which coefficients move: `ordinary` (everything), `ordinary_no_replay`, `frozen_old` (a coefficient is "old" if its feature row *and* class column already existed; old coefficients do not move), `small_old_lr` (old coefficients move at $0.1\times$ the step), and two null-space rules, `strategy_b` and `strategy_b_new`, that keep the logits at three stored reference inputs per old class fixed to first order. *Learning rates* may be a single shared value or one value per degree group $\eta = [\eta_0, \eta_1, \eta_2, \eta_3]$ for degrees 0–3.

4 Benchmark protocol and experimental design

4.1 Protocol

Table 1 lists the protocol (version 1.0). It is defined independently of any algorithm and is implemented in the benchmark package, which depends on the polynomial package but never modifies it.

Let $T = 5$ and $R_{i,j}$ be the accuracy on the test examples of experience j after training through experience i, predicting by argmax over the classes introduced through i. Final mean accuracy is $\mathrm{Acc} = \frac{1}{T}\sum_j R_{T-1,j}$ and mean forgetting

$$\mathrm{Forg} = \frac{1}{T-1}\sum_{j=0}^{T-2}\Big(\max_{j\le i\le T-1} R_{i,j} - R_{T-1,j}\Big), \tag{2}$$

which excludes the last experience, as in the original implementation. We also report *newest-experience accuracy* $R_{T-1,T-1}$ to expose stability bought by not learning. Seeds change shuffling, replay selection and (for the MLP) initialisation, *not* the data or the projection; the reported spread therefore understates dataset-level variability.

4.2 Competitor

The competitor is an experience-replay MLP, $16 \to H \to 10$ with one ReLU layer (or a linear head for $H = \varnothing$), He initialisation, float32, plain SGD on batch-mean cross-entropy, a masked softmax over introduced classes and *the same memory and class-balanced sampler as the polynomial method* (the benchmark re-implements the core's batching algorithm because the core exposes it only inside

its training routine). For every seed both algorithms are given the same experience plan, and the paired-comparison code asserts equality of plan fingerprints before computing any difference. The ER-MLP search space is $H \in \{\varnothing, 64, 128, 256, 512\}$ and $\eta \in \{0.01, 0.03, 0.1, 0.3, 1, 3, 10\}$ (35 configurations; no momentum, weight decay or learning-rate schedule).

4.3 Selection, criteria and the analysis tool

Every configuration is scored on the validation split with selection seeds, then run on the test split with the evaluation seeds. The polynomial study explored 216 three-epoch candidates: 6 update rules × 2 sampling policies × 18 learning-rate vectors (the shared-rate grid $\{0.01, \ldots, 30\}$, one degree group scaled by $0.25, 0.5, 2, 4$ at a time, and one combined vector). Ten-epoch re-runs of the shared and selected vector of each cell exist for the test seeds only.

A script (`tools/analyze_per_degree_study.py`) reads the study's result files and logs and applies four criteria, each reporting the other two metrics and the configuration: highest accuracy; lowest forgetting; smallest time; and a *trade-off*, defined as the Euclidean distance to the ideal point after min–max normalising accuracy (up), forgetting (down) and time (down) over the candidate pool. The trade-off definition is a convention; the script also reports the winner under other weightings. Because "lowest forgetting" and "smallest time" are degenerate without a quality constraint, the script additionally reports them within an *accuracy floor*: candidates within 2 points of the best accuracy. Two ranking bases are produced and kept apart: **validation** (rank on validation, report test; an honest estimate) and **test** (rank on the test seeds, the 10-epoch re-runs included; hindsight, optimistic). The same code ranks the ER-MLP candidates.

4.4 Cost measurement and its limits

Time is the wall-clock time of one complete run (five experiences plus in-loop evaluation) as reported by each adapter, identical in definition for both algorithms, measured in one process on one CPU core. The polynomial per-run time excludes the one-time construction of the degree-3 features for the 70 000 images (15.8 s here), which the ER-MLP does not need. The polynomial study's own logs record times on a different, slower machine (t_{log}); every comparison against the ER-MLP therefore uses times re-measured here (t_{here}). Parameter counts are those of the full 10-way head (including columns of classes not yet introduced), and the stored memory is identical for all replay methods.

5 Results

5.1 Best settings by criterion (RQ3)

Table 2 gives the analysis output for the polynomial method. Ranked on validation, the highest-accuracy configuration is `frozen_old` with class-balanced sampling and $\eta = [2.5, 2.5, 10, 10]$, at 77.8% accuracy and 10.5% forgetting on the test seeds. The lowest-forgetting configuration (`strategy_b`) reaches 8.5% forgetting at 75.9% accuracy, but the null-space rules take roughly 2.5–3 times as long as the other rules. The unconstrained smallest-time winner is degenerate: a no-replay model with 22.1% accuracy. Low forgetting is also easy to obtain by learning little (Fig. 1b), which is why newest-experience accuracy is reported beside it. Within the accuracy floor, the cheapest configuration is plain `ordinary` replay with a single rate of 3 (76.9%, 9.7% forgetting).

The trade-off winner is fragile. Ranked on validation, it is `ordinary / class_balanced / shared / e3` for equal weights, accuracy, forgetting and time weighted double, but `ordinary / class_balanced / combined / e3` when time is ignored. Ranked in hindsight on test, it varies across weightings among `ordinary / class_balanced / d2x2 / e3`, `small_old_lr / class_balanced / d1x0.5 / e3` and `small_old_lr / class_balanced / d1x0.25 / e3`. We conclude that a "best trade-off" is only meaningful relative to a stated weighting, and that the candidates differ by less than the seed-to-seed spread in accuracy (about 0.3–0.5 points), so they are practically exchangeable.

Criterion	Update rule	Sampling	η (degrees 0–3)	Acc. (%)	Forg. (%)	Newest (%)	$t_{\log}$ (s)	t_{here} (s)
Ranked on validation (selection seeds 0–2); test seeds 3–9 reported								
Highest accuracy	`frozen_old`	`class_balanced`	[2.5, 2.5, 10, 10]	77.8±0.4	10.5±1.1	72.2	5.00	0.79
Lowest forgetting	`strategy_b`	`class_balanced`	[3, 3, 0.75, 3]	75.9±0.2	8.5±0.5	65.6	7.26	2.29
Smallest time	`ordinary_no_replay`	`shuffled`	[0.01, 0.01, 0.01, 0.01]	22.1±0.0	86.9±0.0	86.9	3.06	0.71
Best trade-off	`ordinary`	`class_balanced`	[3, 3, 3, 3]	76.9±0.3	9.7±0.8	70.6	3.53	0.91
Lowest forgetting (floor)	`strategy_b`	`class_balanced`	[12, 0.75, 0.75, 6]	76.9±0.3	9.0±0.8	69.3	7.20	2.50
Smallest time (floor)	`ordinary`	`class_balanced`	[3, 3, 3, 3]	76.9±0.3	9.7±0.8	70.6	3.53	0.91
Best trade-off (floor)	`frozen_old`	`class_balanced`	[2.5, 2.5, 10, 10]	77.8±0.4	10.5±1.1	72.2	5.00	0.79
Ranked on the test seeds themselves (hindsight; optimistic)								
Highest accuracy (10 ep.)	`frozen_old`	`class_balanced`	[2.5, 2.5, 10, 10]	78.6±0.5	11.9±1.1	75.6	6.96	1.57
Lowest forgetting	`strategy_b`	`class_balanced`	[3, 3, 0.75, 3]	75.9±0.2	8.5±0.5	65.6	7.26	2.29
Smallest time	`ordinary_no_replay`	`shuffled`	[0.04, 0.01, 0.01, 0.01]	17.9±0.1	91.2±0.0	86.6	3.00	0.71
Best trade-off	`ordinary`	`class_balanced`	[3, 3, 6, 3]	77.2±0.4	10.1±0.9	71.3	3.50	0.84
Lowest forgetting (floor)	`strategy_b`	`class_balanced`	[12, 0.75, 0.75, 6]	76.9±0.3	9.0±0.8	69.3	7.20	2.50
Smallest time (floor)	`ordinary`	`class_balanced`	[3, 3, 6, 3]	77.2±0.4	10.1±0.9	71.3	3.50	0.84
Best trade-off (floor)	`small_old_lr`	`class_balanced`	[10, 2.5, 10, 10]	78.1±0.3	10.3±1.1	72.8	3.64	0.81

Table 2: Polynomial CIL: the four selection criteria under two ranking bases (output of the analysis script). "Floor" restricts the pool to candidates within 2 points of the best accuracy. Accuracy and forgetting: mean ± s.d. over test seeds 3–9; "Newest" is accuracy on the last experience; $t_{\log}$: time recorded in the study logs on the original machine; t_{here}: time re-measured on the machine used for the comparison with ER-MLP.

For the remainder we call the validation-selected highest-accuracy setting the *polynomial headline setting*.

5.2 Polynomial CIL versus experience replay (RQ1)

Table 3 places both methods on the same protocol, selection procedure and machine. The polynomial headline setting attains 77.8% ± 0.4 accuracy with 10.5% forgetting; the validation-selected ER-MLP (H=512, η=1) attains 75.7% ± 1.1 with 17.0% forgetting. That ER-MLP is more plastic (newest-experience accuracy 77.4% versus 72.2%) and forgets more. Against the ER-MLP chosen in hindsight on the test seeds (H=512, η=0.3, 76.0%, 13.2% forgetting) the polynomial setting is still ahead, which makes the comparison conservative in the competitor's favour.

Table 4 gives paired differences, which are meaningful because both methods see the same experiences and exemplars for each seed. The polynomial setting is better in accuracy by +2.1 points (95% CI +1.1 to +3.1) against the validation-selected ER-MLP and by +1.8 points (+1.0 to +2.6) against the hindsight-best ER-MLP, in 7 of 7 seeds, and it forgets less in every seed. The CIs are t-intervals over 7 seeds that vary only shuffling, replay selection and initialisation: they quantify optimiser noise, not dataset variability, and a sign test over seven seeds cannot go below $p = 2/2^7 \approx 0.016$. We therefore read the result as a *consistent small advantage on this protocol*, not as established superiority.

Cost does not separate the methods clearly. Per-run time is 0.79 s for the polynomial setting and 0.71 s for the ER-MLP with H=512, but the ER-MLP with H=256 reaches 75.7% in 0.29 s with 6,922 parameters (polynomial: 9,690). Counting the one-time feature construction of 15.8 s, the polynomial method is clearly the more expensive of the two. The extra accuracy of +2.1 points over the H=256 model therefore comes with about 1.4× the parameters and a fixed up-front cost.

5.3 Where does the effect come from? (RQ2)

Capacity. A degree-1 polynomial model is a linear softmax classifier and, as expected, matches the ER-MLP with a linear head (66.8% versus 66.8%, forgetting 13.1% versus 13.1%), which also serves as a sanity check of the two implementations. Raising the degree to 3 from the start gives 77.5%, an increase of about 11 points. Polynomial capacity is the dominant factor.

Gradual expansion. Expanding from degree 1 to 3 over the sequence does *not* beat starting at degree 3. With a shared rate, the expanding schedule with plain replay gives 76.9% accuracy and 9.7% forgetting; degree 3 from the start, with its rate chosen on validation, gives 77.5% and 12.5% forgetting. Expansion is thus associated with less forgetting (by about 2.8 points) and lower

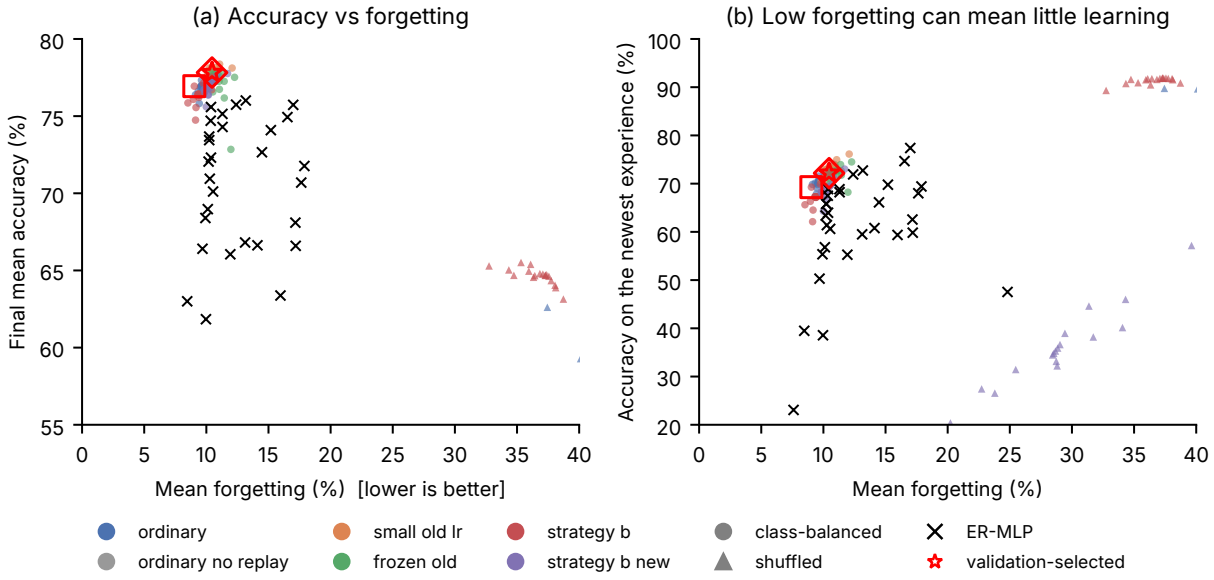


Figure 1: (a) Final accuracy versus forgetting and (b) newest-experience accuracy versus forgetting for 216 polynomial candidates and 35 ER-MLP configurations (test seeds). 91 polynomial points below 55% accuracy (mainly no-replay models) are outside panel (a). Red outlines: validation-selected polynomial settings. In panel (b), configurations with the lowest forgetting have the lowest newest-experience accuracy: they forget little because they learned little.

Setting	Configuration	Params	Memory	Acc. (%)	Forg. (%)	Newest (%)	Time (s)
Polynomial (expanding degrees $1,2,3,3,3$*)*							
Highest accuracy (val.)	`frozen_old`, $\eta = [2.5,2.5,10,10]$	9,690	1,600	77.8±0.4	10.5±1.1	72.2	0.79
Best trade-off (val., literal)	`ordinary`, $\eta = [3,3,3,3]$	9,690	1,600	76.9±0.3	9.7±0.8	70.6	0.91
Lowest forgetting (val., floor)	`strategy_b`, $\eta = [12,0.75,0.75,6]$	9,690	1,600	76.9±0.3	9.0±0.8	69.3	2.50
Degree 1 throughout (val.-tuned η)	`ordinary`, $\eta = 3$	170	1,600	66.8±0.4	13.1±0.8	59.0	0.16
Degree 3 from the start (val.-tuned η)	`ordinary`, $\eta = 10$	9,690	1,600	77.5±0.6	12.5±1.8	74.2	1.02
No replay (core-selected η)	`ordinary_no_replay`, $\eta = [0.0025,0.01,0.02,0.04]$	9,690	0	25.6±0.0	82.4±0.0	87.0	0.76
ER-MLP (one hidden ReLU layer, plain SGD, replay)							
Highest accuracy (val.)	$H = 512$, $\eta = 1$	13,834	1,600	75.7±1.1	17.0±2.2	77.4	0.71
Best trade-off (val., literal)	$H = 64$, $\eta = 0.1$	1,738	1,600	72.3±0.3	10.4±1.0	64.0	0.19
Best trade-off (val., floor)	$H = 256$, $\eta = 0.3$	6,922	1,600	75.7±0.5	12.4±1.4	71.9	0.29
Lowest forgetting (val., floor)	$H = 256$, $\eta = 0.1$	6,922	1,600	74.7±0.3	10.3±1.4	67.5	0.32
Highest accuracy (hindsight, test)	$H = 512$, $\eta = 0.3$	13,834	1,600	76.0±0.9	13.2±1.9	72.7	0.59
Linear head (val.-tuned η)	linear head, $\eta = 0.3$	170	1,600	66.8±0.2	13.1±0.5	59.5	0.12
No replay	$H = 512$, $\eta = 1$	13,834	0	18.8±0.1	96.5±0.2	93.9	0.67

Table 3: Head-to-head on identical protocol, replay memory, seeds and machine (test seeds 3–9, 3 epochs). "val." = ranked on validation; "floor" = within 2 points of the best accuracy; "hindsight" = ranked on the test seeds (optimistic). Params: parameters of the 10-way head. Memory: stored exemplars at the end.

newest-experience accuracy (70.6% versus 74.2%), that is, more stability and less plasticity, with no accuracy gain. This comparison is not perfectly balanced: the fixed-capacity model was tuned only over the shared-rate grid and without the update rules, which require "old" coefficients to exist.

Replay. Without replay the polynomial model reaches only 25.6% (82.4% forgetting) and the ER-MLP 18.8%; both predict almost only the newest classes (newest-experience accuracy $\geq 87.0\%$).

Sampling policy, update rule and per-degree rates. Table 5 shows each update rule under both sampling policies. With class-balanced sampling the five replay-based rules lie within a narrow band (about 76–78%), and the learning-rate vector chosen per degree changes accuracy by between -0.4 and +0.5 points relative to a shared rate. Under shuffled sampling accuracy collapses (median 54.0% versus 77.1%), with large seed-to-seed variance. In that regime the update rules do differ: `strategy_b` (64.8%) beats `ordinary` (56.8%) and `frozen_old` (46.2%) with a shared rate. A reading consistent with these numbers, though not proved by them, is that much of what the update rules and large per-degree rates achieve under shuffled sampling is compensation for class

Polynomial (validation-selected) − competitor	Δ acc. (pts)	95% CI	Δ forg. (pts)	95% CI
ER-MLP, validation-selected ($H = 512$, $\eta = 1$)	+2.1	[+1.1, +3.1]	-6.5	[-8.3, -4.8]
ER-MLP, hindsight-best on test ($H = 512$, $\eta = 0.3$)	+1.8	[+1.0, +2.6]	-2.7	[-4.1, -1.4]
ER-MLP, best trade-off within accuracy floor ($H = 256$, $\eta = 0.3$)	+2.1	[+1.5, +2.6]	-1.9	[-2.9, -1.0]

Table 4: Paired differences (polynomial headline setting minus competitor) over the seven test seeds. Positive accuracy difference and negative forgetting difference favour the polynomial method. Seeds do not vary the data, so intervals understate dataset-level uncertainty.

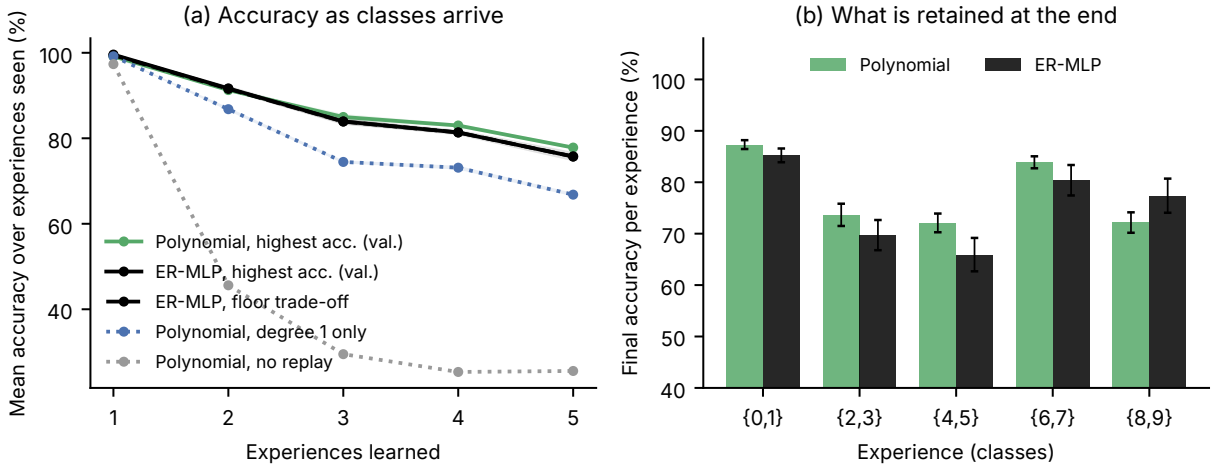


Figure 2: (a) Mean accuracy over experiences seen, after each experience (mean ± s.d. over seeds). (b) Final accuracy on each experience. The polynomial setting retains the middle experiences better; the ER-MLP is better on the newest one (77.4% versus 72.2%).

imbalance (about 5 900 current examples versus 200 stored per old class in the last experience), which class-balanced sampling removes directly. The headline setting is not better than the shared-rate cells (77.6% for `frozen_old`, 78.0% for `small_old_lr`); the differences among these cells are within seed noise, so the per-degree vector is not what separates the polynomial method from the ER-MLP, and the ranking of the top polynomial cells is not resolved by this study.

5.4 Training budget (RQ4)

With ten epochs (Table 6, no re-selection) the polynomial setting improves to 78.6% (forgetting 11.9%), whereas the ER-MLPs do not improve in accuracy and forget more (for example, forgetting rises from 12.4% to about 17% for H=256). The best ER-MLP at ten epochs reaches 76.0%, so the gap of roughly 2.6 points persists. Time grows two- to threefold for both. Because rates were selected for three epochs, a re-tuned ER-MLP might do better at ten; we did not test this.

5.5 Does validation selection transfer? (RQ5)

Across the 216 validation candidates, validation and test accuracy are almost perfectly rank-correlated (Spearman $\rho = 0.99$; forgetting $\rho = 0.99$), but this is driven by the wide range that includes no-replay and unstable candidates; among the 60 best candidates it falls to $\rho = 0.86$. Test accuracy is on average +1.0 points higher than validation accuracy, plausibly because the validation models are fitted to 90% of the training data and the two splits are not equally difficult; we did not investigate this. The validation-selected setting is 0.5 points below the best three-epoch candidate in hindsight, and the hindsight best overall (ten epochs, 78.6%) differs from the validation-selected one only through the extra epochs. The optimism of selecting on test is thus small here, but it is not zero, and it is why we base all claims on the validation ranking.

		Shared rate η			Per-degree vector (validation-selected)			
Update rule	Sampling	η	Acc. (%)	Forg. (%)	η (deg. 0–3)	Acc. (%)	Forg. (%)	t (s)
`ordinary`	`class_balanced`	3	76.9±0.3	9.7±0.8	[0.75, 1.5, 1.5, 6]	77.3±0.3	9.6±0.8	0.79
`ordinary`	`shuffled`	30	56.8±7.2	43.7±8.8	[30, 30, 30, 120]	62.6±8.1	37.4±11.0	0.87
`ordinary_no_replay`	`class_balanced`	0.01	22.1±0.0	86.5±0.0	[0.0025, 0.01, 0.02, 0.04]	25.6±0.0	82.4±0.0	0.76
`ordinary_no_replay`	`shuffled`	0.01	22.1±0.0	86.9±0.0	[0.0025, 0.0025, 0.01, 0.04]	26.4±0.0	81.5±0.0	0.72
`small_old_lr`	`class_balanced`	10	78.0±0.4	10.5±1.3	[20, 10, 10, 10]	77.8±0.5	10.1±2.0	0.87
`small_old_lr`	`shuffled`	10	54.8±5.0	48.7±6.1	[2.5, 40, 20, 40]	58.5±6.5	42.9±7.3	0.74
`frozen_old`	`class_balanced`	10	77.6±0.5	10.8±1.5	[2.5, 2.5, 10, 10]	77.8±0.4	10.5±1.1	0.79
`frozen_old`	`shuffled`	30	46.2±7.1	55.7±8.0	[15, 30, 30, 30]	47.6±6.3	55.1±6.3	0.76
`strategy_b`	`class_balanced`	3	76.4±0.2	9.4±0.7	[12, 0.75, 0.75, 6]	76.9±0.3	9.0±0.8	2.50
`strategy_b`	`shuffled`	30	64.8±0.8	37.1±1.3	[120, 120, 120, 60]	65.3±1.0	32.7±2.1	2.47
`strategy_b_new`	`class_balanced`	10	77.1±0.2	10.3±0.7	[5, 5, 2.5, 10]	76.7±0.3	9.8±0.7	2.65
`strategy_b_new`	`shuffled`	0.03	53.5±2.0	28.7±2.3	[0.12, 0.12, 0.03, 0.0075]	50.5±3.3	16.7±1.7	2.54

Table 5: Update rule × sampling policy: shared-rate reference versus the validation-selected per-degree vector (test seeds 3–9, 3 epochs; t re-measured here for the selected vector).

Method	Configuration	Acc. 3 ep.	Acc. 10 ep.	Forg. 3 ep.	Forg. 10 ep.	Time 3 ep.	Time 10 ep.
Polynomial	`frozen_old`, $\eta = [2.5, 2.5, 10, 10]$	77.8±0.4	78.6±0.5	10.5±1.1	11.9±1.1	0.79	1.57
ER-MLP	$H = 512$, $\eta = 1$	75.7±1.1	74.2±1.3	17.0±2.2	23.0±2.0	0.71	2.01
ER-MLP	$H = 512$, $\eta = 0.3$	76.0±0.9	76.0±0.7	13.2±1.9	17.6±1.2	0.59	1.94
ER-MLP	$H = 256$, $\eta = 0.3$	75.7±0.5	75.6±0.6	12.4±1.4	16.8±1.2	0.29	1.05

Table 6: Three versus ten epochs per experience, without re-selection (test seeds 3–9).

6 Discussion

Is polynomial CIL "an answer"? On this protocol it is a competitive and modestly better answer than a replay MLP of comparable size, with lower forgetting and a consistent paired advantage across seeds. It is not demonstrated to be an answer *for the reason it was proposed*. The method's three distinguishing ingredients (incremental expansion, protective update rules, per-degree learning rates) contribute little or nothing measurable once class-balanced replay is used; the strong effects are polynomial capacity, replay and balanced sampling. A fair summary is that a degree-3 polynomial head on a 16-dimensional projection is a well-conditioned, cheap-to-train classifier that suits replay-based CIL, and that gradual expansion trades some plasticity for stability.

Why might the polynomial head forget less? The ER-MLP adapts a shared hidden representation to every experience, whereas a polynomial head with a fixed feature map changes only a linear readout of fixed features; this is a plausible reason for lower forgetting and lower plasticity (Fig. 2b), but the present study does not isolate it (for example by freezing the MLP's hidden layer or by adding random-feature baselines of matching width).

What "best" means. The criteria produce four different winners, two of which are degenerate (Fig. 1b). A benchmark should therefore report a Pareto set with newest-experience accuracy, as the accompanying results do, and not a single scalar; the trade-off score is useful for presenting one default, provided its weighting is stated.

Open questions. Does the advantage persist with higher-dimensional inputs, where the number of monomials explodes and the 16-dimensional projection is not available? Is it a property of the head or of the fixed projection? How do stronger CIL methods (iCaRL-style exemplar classifiers, gradient-projection methods) compare? Would a tuned learning-rate schedule or weight decay remove the ER-MLP's disadvantage? Would a different task order or a different number of classes per experience change the ranking of update rules?

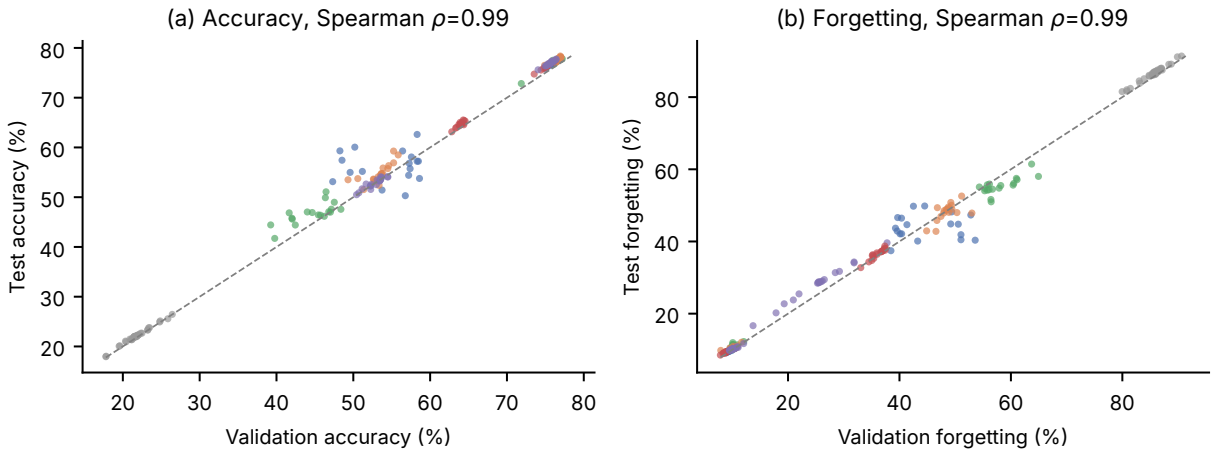


Figure 3: Validation versus test (a) accuracy and (b) forgetting for all three-epoch polynomial candidates, coloured by update rule. The dashed line is $y = x$.

Threats to validity and limitations

- **One dataset and a weak input representation.** Split MNIST on a 16-dimensional random projection yields about 78% accuracy; absolute numbers are not comparable to results using full-resolution inputs and deeper networks. Additional datasets will be needed if reviewers ask for them.
- **One competitor family.** Only an ER-MLP was run. EWC, LwF and other methods were not, so the comparison does not establish a position among CIL methods.
- **Unequal search effort.** The polynomial method was tuned over 216 candidates (including structural choices), the ER-MLP over 35 (width and rate). The ER-MLP's validation-selected configuration sits at the upper edge of the width grid (H=512; its rate is interior), so a wider search could raise it.
- **Seed variance only.** Seeds do not vary the data split or projection.
- **Timing.** Single machine, one core, sub-second to few-second runs; excluding polynomial feature construction favours the polynomial method.
- **Numerical reproducibility.** Re-running 28 polynomial settings against the stored core results reproduced 21 of them bit-for-bit. All class-balanced settings agree to $\leq 1.2 \times 10^{-4}$; the largest differences ($\leq 9.6e-02$) occur for shuffled sampling with rates ≥ 30, where training is unstable and floating-point differences between machines are amplified. Conclusions rest on the class-balanced settings.
- **Fixed-capacity ablation.** It was tuned with a shared rate only.

7 Conclusion

Under one controlled protocol, polynomial CIL with class-balanced replay is a small but consistent improvement over a tuned experience-replay MLP in accuracy and forgetting, at comparable per-run cost once features are available. The improvement is attributable chiefly to polynomial capacity and balanced replay, not to incremental expansion, update rules or per-degree learning rates, which the ablations leave unsupported as sources of accuracy. The study's main methodological lesson is that single-number "best" claims (lowest forgetting, smallest time, best trade-off) are fragile or degenerate and should be replaced by validation-based Pareto reporting.

A Reproducibility

- **Core package** `polynomial_cl`: commit `b0b917ee240db7a7ea7d78cb4bf21754e99d554b` of the checkout supplied for this study, with uncommitted working-tree changes (21 modified files). The benchmark's imports were updated to the refactored core API (`polynomial_cl.continual`, `polynomial_cl.data`); the core was not modified. After this change the adapter reproduces the stored seed-3 result of the combined per-degree setting exactly (accuracy 0.7734281931972611, forgetting 0.09086581818480258).
- **Stored study results**: produced by `experiments/run_per_degree_study.sh` of the core (selection seeds 0–2, test seeds 3–9).
- **Analysis** (wall time is not stored in the result JSON; it is read from the logs and each log line is cross-checked against the JSON run before use):

```
python tools/analyze_per_degree_study.py \
  --results-dir <core>/results/per_degree \
  --out-json results/analysis/per_degree_analysis.json \
  --out-md   results/analysis/per_degree_analysis.md \
  --out-csv  results/analysis/per_degree_candidates.csv
```

- **Comparison** (resumable; every run, failed ones included, is appended to `runs.jsonl`):

```
python tools/run_comparison_study.py \
  --analysis results/analysis/per_degree_analysis.json \
  --core-results <core>/results/per_degree \
  --out-dir results/comparison
```

- **Paper assets** (all tables, figures and quoted numbers are generated here):

```
python tools/build_paper_assets.py \
  --analysis results/analysis/per_degree_analysis.json \
  --comparison results/comparison --out paper
```

References

[1] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[2] G. M. van de Ven and A. S. Tolias. Three scenarios for continual learning. arXiv:1904.07734, 2019.

[3] Y.-C. Hsu, Y.-C. Liu, A. Ramasamy and Z. Kira. Re-evaluating continual learning scenarios: a categorization and case for strong baselines. arXiv:1810.12488, 2018.

[4] A. Chaudhry et al. On tiny episodic memories in continual learning. arXiv:1902.10486, 2019.

[5] S.-A. Rebuffi, A. Kolesnikov, G. Sperl and C. H. Lampert. iCaRL: incremental classifier and representation learning. In *CVPR*, 2017.

[6] A. Prabhu, P. H. S. Torr and P. K. Dokania. GDumb: a simple approach that questions our progress in continual learning. In *ECCV*, 2020.

[7] A. Chaudhry, P. K. Dokania, T. Ajanthan and P. H. S. Torr. Riemannian walk for incremental learning: understanding forgetting and intransigence. In *ECCV*, 2018.

[8] M. De Lange et al. A continual learning survey: defying forgetting in classification tasks. *IEEE TPAMI*, 44(7):3366–3385, 2022.

[9] N. Díaz-Rodríguez, V. Lomonaco, D. Filliat and D. Maltoni. Don't forget, there is more than forgetting: new metrics for continual learning. arXiv:1810.13166, 2018.

[10] S. Farquhar and Y. Gal. Towards robust evaluations of continual learning. arXiv:1805.09733, 2018.

[11] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.

[12] F. Zenke, B. Poole and S. Ganguli. Continual learning through synaptic intelligence. In *ICML*, 2017.

[13] Z. Li and D. Hoiem. Learning without forgetting. *IEEE TPAMI*, 40(12):2935–2947, 2018.

[14] D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. In *NeurIPS*, 2017.

[15] M. Farajtabar, N. Azizan, A. Mott and A. Li. Orthogonal gradient descent for continual learning. In *AISTATS*, 2020.

[16] G. Saha, I. Garg and K. Roy. Gradient projection memory for continual learning. In *ICLR*, 2021.

[17] Z. Mai et al. Online continual learning in image classification: an empirical survey. *Neurocomputing*, 469:28–51, 2022.

[18] A. G. Ivakhnenko. Polynomial theory of complex systems. *IEEE Trans. Systems, Man, and Cybernetics*, 1(4):364–378, 1971.

[19] Y.-W. Chang, C.-J. Hsieh, K.-W. Chang, M. Ringgaard and C.-J. Lin. Training and testing low-degree polynomial data mappings via linear SVM. *JMLR*, 11:1471–1490, 2010.

[20] G. G. Chrysos et al. Deep polynomial neural networks. *IEEE TPAMI*, 44(8):4021–4034, 2022.